



Systemes paralleles et distribues

Projet : Parallel Ant Foraging

Javier andres TARAZONA JIMENEZ

Jose Daniel CHACON

Encadrant : JUVIGNY Xavier

École Nationale Supérieure des Techniques Avancées (ENSTA)
Majeure STIC
Mars 2026

Table des matières

1	Introduction	3
1.1	Objectifs de la simulation	3
2	Analyse du problème - Baseline	4
2.1	État de l'art	4
2.2	Profilage et identification des goulots d'étranglement	4
2.3	Résultats	5
2.3.1	Analyse des Résultats	6
3	Vectorisation	8
3.1	Stratégie de Vectorisation	8
3.1.1	Motivation	8
3.1.2	Du layout AoS au layout SoA	8
3.1.3	Maintien du baseline : un binaire, deux layouts	8
3.1.4	Choix d'implémentation : types compacts et seed par fourni	8
3.1.5	Application dans le noyau : <code>advance_time_soa</code>	9
3.1.6	Découplage de <code>SDL_Point</code> hors hot loop et adaptation du rendu	9
3.1.7	Validation et protocole de comparaison	9
3.2	Résultat du Profilage	9
3.3	Analyse des Résultats	10
4	Parallélisation en mémoire partagée (OpenMP)	12
4.1	Stratégie retenue	12
4.1.1	Objectif	12
4.1.2	Choix d'architecture	12
4.1.3	Boucles parallélisées	12
4.1.4	Correctness et absence de data races	12
4.1.5	Réduction sparse et robustesse d'ordre	12
4.1.6	Profilage OpenMP-safe	13
4.2	Protocole expérimental	13
4.3	Résultats	13
4.3.1	Calcul explicite du speedup mesuré	13
4.3.2	Vérifications empiriques de correctitude	16
4.4	Analyse et discussion des résultats	16
5	Comparaison des campagnes (ancienne machine vs campagne locale)	18

5.1	Objectif	18
5.2	Comparaison synthétique	18
5.3	Interprétation des écarts	18
5.4	Ce qui reste stable malgré les écarts	18

1 Introduction

1.1 Objectifs de la simulation

L'objectif principal de la simulation est de modéliser le problème de fourragement d'une colonie de fourmis artificielles sur un terrain 2D hétérogène, afin d'identifier des trajets efficaces entre la fourmilière et une source de nourriture. Le cadre retenu suit un modèle d'intelligence en essaim où des agents simples produisent un comportement collectif émergent à partir de règles locales.

Le modèle vise à reproduire une coordination indirecte entre agents via deux types de phéromones : un phéromone d'exploration vers la nourriture (V_1) et un phéromone de retour vers le nid (V_2). Chaque fourmi évolue avec deux états (*unloaded*/*loaded*) et applique une politique exploration–exploitation pilotée par le paramètre ε , avec des transitions d'état lors de l'arrivée sur la nourriture ou sur la fourmilière.

L'environnement a pour objectif de représenter une difficulté de déplacement réaliste : chaque cellule de la carte possède un coût de traversée dans $[0, 1]$, et les zones non franchissables sont traitées comme obstacles. Le paysage est généré de manière fractale afin d'introduire des gradients de difficulté non uniformes et de contraindre la dynamique des déplacements.

Enfin, la simulation poursuit un objectif d'évaluation fonctionnelle au niveau de la colonie : la quantité de nourriture rapportée au nid sert d'indicateur global de performance. Ce cadre constitue la base expérimentale du projet, avant les étapes d'optimisation algorithmique (vectorisation et parallélisation) décrites dans le contexte général.

2 Analyse du problème - Baseline

2.1 État de l'art

L'implémentation actuelle initialise d'abord l'environnement dans `fractal_land.cpp` en construisant une carte fractale par raffinement récursif de sous-grilles. Dans `ant_simu.cpp`, cette carte est ensuite normalisée pour ramener les altitudes dans l'intervalle $[0, 1]$, ce qui fixe le coût de déplacement utilisé par les fourmis pendant la simulation.

La population est ensuite instanciée avec un nombre fixe de fourmis, des positions initiales pseudo-aléatoires et des paramètres globaux du modèle (*exploration*, *bruit*, *évaporation*). La structure `pheromone` (dans `pheromone.hpp`) maintient deux valeurs de phéromones par cellule, un tampon de mise à jour, ainsi que des bordures fantômes marquées à -1 pour empêcher les traversées hors domaine.

À chaque itération, la boucle principale de `ant_simu.cpp` traite les événements SDL puis exécute `advance_time`. Cette routine appelle `ant::advance` pour chaque agent, applique l'évaporation globale des phéromones (`do_evaporation`) puis valide l'état suivant de la carte (`update`). L'évolution individuelle d'une fourmi, définie dans `ant.cpp`, combine un choix local de voisin (aléatoire ou guidé par le maximum de phéromone), l'accumulation d'un budget de mouvement (`consumed_time < 1`), le marquage local des phéromones et la mise à jour de l'état chargé/non chargé.

Le rendu de l'état simulé est assuré par `render.cpp` et `window.cpp`. Le rendu affiche le terrain, la position des fourmis, la distribution des deux phéromones et la courbe de nourriture cumulée. La couche fenêtre/render SDL est construite avec mécanismes de repli (*fallback*) pour conserver l'exécution même lorsque certaines options matérielles ne sont pas disponibles.

2.2 Profilage et identification des goulots d'étranglement

Le profilage a pour objectif d'isoler les coûts dominants de la simulation avant toute optimisation, en particulier avant les étapes de vectorisation et de parallélisation. Nous présentons ici la définition des blocs à mesurer, instrumentation temporelle, protocole d'exécution reproductible et structure des résultats attendus. Cette section décrit donc la méthode de profilage et les objets techniques mobilisés, sans présenter de résultats chiffrés.

TABLE 1 – Découpage des parties mesurées pour le profilage de la simulation

ID	Nature	Rôle	Périmètre
P0	Initialisation	Génération fractale du terrain	One-shot (démarrage)
P1	Initialisation	Normalisation des coûts du terrain	One-shot (démarrage)
P2	Initialisation	Création et placement de la population	One-shot (démarrage)
K0	Noyau simulation	Coût total du noyau par itération	Itératif
K1	Noyau simulation	Avance globale de toutes les fourmis	Itératif
K2	Noyau simulation	Logique interne hors marquage phéromones	Itératif
K3	Noyau simulation	Marquage local des phéromones	Itératif
K4	Noyau simulation	Évaporation globale des phéromones	Itératif
K5	Noyau simulation	Validation/swap de la carte de phéromones	Itératif
R0	Rendu SDL	Affichage (terrain, fourmis, phéromones, courbe)	Itératif, hors noyau
E0	Événements SDL	Traitement de <code>SDL_PollEvent</code>	Itératif, hors noyau

Les relations structurelles utilisées pour l’analyse des goulots sont explicites :

$$K0 = K1 + K4 + K5$$

$$K1 = K2 + K3 + \text{reste}(K1)$$

Les blocs $P0$, $P1$ et $P2$ sont des coûts *one-shot* d’initialisation et ne doivent pas être mélangés aux métriques par itération. Les blocs $R0$ (rendu SDL) et $E0$ (gestion des événements SDL) sont mesurés séparément et traités comme non parallélisables dans l’analyse du noyau de calcul.

La stratégie instrumentale côté C++ s’appuie sur `timing_profile.hpp` et `timing_profile.cpp`. Le profilage est désormais *instance-based* (plus d’état mutable global partagé) : chaque exécution benchmark manipule son objet `TimingProfile` et les compteurs d’itération `IterTimingNs`. `timing_profile.hpp` fournit l’horloge monotone (`profile_now_ns`) utilisée pour les frontières de kernels ($K0$, $K1$, $K4$, $K5$), tandis que $K2/K3$ sont accumulés côté boucle agents comme temps de travail (*work-time*) dans les métriques d’itération. Cette organisation sépare proprement coûts d’initialisation, coûts noyau et coûts hors noyau.

L’automatisation est assurée par `scripts/measure_temps.sh`. Le pipeline suit une séquence reproductible : compilation release, puis 5 répétitions d’exécution avec 1200 itérations par répétition, 200 itérations de *warm-up* exclues et 1000 itérations effectivement mesurées, en configuration `OMP_NUM_THREADS=1`. Le script consolide les artefacts méthodologiques (`raw_measurements.csv`, `init_times.csv`, `table_a.csv`, `table_b.csv`, `amdahl_prediction.csv`, `summary.md`) afin de préparer la lecture des goulots. Les artefacts générés sont :

- `raw_measurements.csv` : mesures brutes par répétition (compteurs en ns et valeurs dérivées par itération).
- `init_times.csv` : synthèse des coûts *one-shot* d’initialisation ($P0$, $P1$, $P2$).
- `table_a.csv` : métriques des blocs de profilage (moyenne, dispersion et part relative du noyau).
- `table_b.csv` : indicateurs globaux par répétition (T_{iter} , T_{kernel} , T_{render} , débit).
- `amdahl_prediction.csv` : valeurs théoriques de speedup issues du modèle d’Amdahl à partir de la fraction parallélisable estimée.
- `summary.md` : synthèse lisible du protocole et des tableaux générés.

La méthode de lecture priorise les métriques `ms/iter` et % noyau pour identifier les *hotspots* dominants. Les conclusions doivent d’abord cibler les blocs du noyau ($K0$ – $K5$), puis isoler l’impact des composants hors noyau ($R0$, $E0$) pour éviter les biais d’interprétation.

2.3 Résultats

Les résultats présentés ci-dessous proviennent de la campagne de mesure :

`results/test_time_measurements_local2/aos/latest/summary.md`. La configuration utilisée est la suivante : 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 2 – Temps one-shot d’initialisation

Partie	ms moyen	écart-type
P0	8.222398	1.107389
P1	0.830145	0.541537
P2	0.171107	0.032904

TABLE 3 – Décomposition des coûts (profilage de base)

Partie	ms/iter moyen	écart-type	% kernel
K1	7.583137	0.803269	94.3004
K2	3.278775	0.344471	40.7733
K3	2.662628	0.413677	33.1112
K4	0.447192	0.087824	5.5611
K5	0.010333	0.002728	0.1285
R0	2.190812	0.341754	n/a
E0	0.005376	0.000923	n/a

TABLE 4 – Indicateurs globaux par répétition

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	8.619821	6.832405	1.783345	731806.773612
2	11.077200	8.649231	2.422158	578086.088303
3	9.838528	7.800593	2.032789	640976.879203
4	9.514154	7.527849	1.981301	664200.401132
5	12.138599	9.397280	2.734469	532068.839698
Moyenne	10.237660	8.041472	2.190812	629427.796390
Écart-type	1.234388	0.893450	0.341754	69193.508032

Fraction parallélisable candidate :

$$F = (K1 + K4)/K0 = 0.997839$$

TABLE 5 – Prédiction théorique selon Amdahl

Threads	S(p)
2	1.995687
4	3.974235
8	7.880787
16	15.497644

2.3.1 Analyse des Résultats

Cette sous-section interprète les mesures présentées dans les tableaux 2, 3, 4 et 5, ainsi que les indicateurs F et $S(p)$. L’objectif est d’identifier les priorités de parallélisation à partir des données observées, en séparant clairement ce qui relève des coûts d’initialisation, du noyau de simulation et des bornes théoriques de gain.

1. **Coûts d’initialisation ($P0$ – $P2$).** Dans le tableau 2, $P0$ domine très largement la phase d’initialisation (8.222398 ms), alors que $P1$ (0.830145 ms) et $P2$ (0.171107 ms) restent faibles. Cette hiérarchie est cohérente avec le fait que la génération fractale du terrain est la tâche la plus lourde au démarrage. En revanche, comme $P0$ – $P2$ sont des coûts *one-shot* hors boucle itérative, ils ne constituent pas la priorité principale pour améliorer le temps de simulation par itération.
2. **Priorité de parallélisation au niveau noyau.** D’après le tableau 3, $K1$ représente 94.3004% du temps noyau, ce qui en fait la cible prioritaire. Le raisonnement est direct : réduire $K1$ a

l'impact potentiel maximal sur $K0$, avant $K4$ (évaporation) et $K5$ (mise à jour de la carte de phéromones), dont les contributions relatives sont nettement plus faibles.

3. **Décomposition interne de $K1$: $K2$ avant $K3$.** Toujours dans le tableau 3, $K2$ pèse 40.7733% du noyau contre 33.1112% pour $K3$. L'ordre de priorité est donc de paralléliser d'abord $K2$, puis $K3$. Cela confirme que la logique de déplacement/décision des fourmis est plus coûteuse que le marquage des phéromones.
4. **Stabilité expérimentale inter-répétitions.** Le tableau 4 montre une faible dispersion entre les 5 répétitions ($T_{\text{iter_total}}$, T_{kernel} , T_{render} , fourmis/s), ce qui indique une bonne stabilité de la campagne de mesure. Cette stabilité provient du protocole (phase de *warm-up* exclue et répétitions multiples), et non d'un effet spécifique "à partir de la cinquième itération".
5. **Interprétation de la fraction parallélisable F .** La valeur $F = 0.998797$, proche de 1, n'est pas anormale dans ce cadre : la définition $F = (K1 + K4)/K0$ inclut les blocs dominants du noyau et exclut les parties hors noyau ($R0$, $E0$). Cette valeur doit toutefois rester interprétée comme une fraction candidate théorique pour le noyau, et non comme une garantie de speedup réel, car les surcoûts de synchronisation, d'accès mémoire et d'équilibrage peuvent réduire le gain observé.
6. **Lecture correcte de la prédiction d'Amdahl.** Le tableau 5 montre un potentiel de gain important avec l'augmentation du nombre de threads. En particulier, $S(16) = 15.497644$ correspond à un facteur multiplicatif d'environ $\times 15.7$, et non à 15%. Cette tendance soutient clairement la stratégie de parallélisation ; la suite du projet doit donc viser d'abord $K1$, puis $K4$, pour se rapprocher de ce plafond théorique.

En synthèse, les deux *hotspots* centraux sont $K1$ au niveau global et $K2$ à l'intérieur de $K1$. Les mesures actuelles établissent une base robuste pour prioriser les optimisations, mais l'écart entre potentiel théorique et performance finale devra être vérifié expérimentalement après implémentation de la parallélisation. La feuille de route est donc cohérente : concentrer l'effort d'abord sur les blocs dominants du noyau afin de maximiser le retour de chaque évolution de code.

3 Vectorisation

3.1 Stratégie de Vectorisation

3.1.1 Motivation

Le point 5.1 de l'énoncé définit la *vectorisation* comme un changement de représentation des agents : au lieu de manipuler une liste d'objets « fourmi », on stocke séparément les attributs (coordonnées, état, graine de générateur pseudo-aléatoire, etc.) dans des tableaux. Une fourmi est alors repérée uniquement par son indice i dans ces tableaux.

Cette étape est motivée par l'analyse *baseline* : le bloc $K1$ (avance de l'ensemble des fourmis) concentre l'essentiel du temps noyau. L'objectif principal est donc d'améliorer l'efficacité mémoire de cette boucle (localité, cache, bande passante) afin de réduire le coût par itération, tout en préparant la suite (parallélisation OpenMP au point 5.2 de la consigne).

3.1.2 Du layout AoS au layout SoA

Dans la version *baseline*, la population est stockée sous la forme d'un `std::vector<ant>` (*Array of Structs*, AoS). Chaque instance `ant` regroupe plusieurs champs hétérogènes (position `SDL_Point`, état, graine RNG, etc.). Lors de l'itération sur toutes les fourmis, le noyau lit/écrit principalement la position, l'état et la graine, mais les lignes de cache chargées contiennent aussi des données non nécessaires au pas courant.

Nous introduisons donc un layout *Structure of Arrays* (SoA), plus favorable à la localité et à la vectorisation au sens « données » du sujet :

$$\text{AoS} : \{(x_i, y_i, s_i, g_i)\}_{i=0}^{N-1} \longrightarrow \text{SoA} : (x[], y[], s[], g[]).$$

Ainsi, les accès séquentiels `x[i]`, `y[i]`, `state[i]`, `seed[i]` deviennent contigus en mémoire, ce qui augmente la proportion de données utiles par ligne de cache et facilite les boucles par indice.

3.1.3 Maintien du baseline : un binaire, deux layouts

Afin de conserver une référence expérimentale stable, le chemin AoS est maintenu et reste le mode d'exécution par défaut. Le programme expose un paramètre de ligne de commande permettant de choisir la représentation :

- `-layout aos` : exécution *baseline* (`std::vector<ant>`);
- `-layout soa` : exécution vectorisée (`AntsSoA`).

Ce choix « 1 binaire, 2 layouts » évite de dupliquer le projet en deux répertoires (et donc de diverger au fil des modifications) et garantit une comparaison directe AoS vs SoA à paramètres identiques.

3.1.4 Choix d'implémentation : types compacts et seed par fourmi

En SoA, nous stockons explicitement les attributs minimaux requis par la dynamique :

- `x[i]`, `y[i]` en `int32_t` (coordonnées);
- `state[i]` en `uint8_t` (0 = *unloaded*, 1 = *loaded*);
- `seed[i]` en `uint32_t` (état RNG local).

Le choix de types compacts réduit l’empreinte mémoire par agent et améliore l’utilisation du cache, ce qui est particulièrement pertinent puisque la boucle $K1$ parcourt plusieurs milliers de fourmis.

Un point important pour la reproductibilité est la gestion de la graine RNG par fourmi. Dans le code AoS *baseline*, le champ `m_seed` est utilisé dans `ant::advance (src/ant.cpp)`. La version SoA rend ce point explicite : chaque fourmi possède sa graine `seed[i]` et les fonctions de génération aléatoire la mettent à jour par référence. Pour garder une comparabilité maximale, la formule du générateur est conservée entre AoS et SoA.

3.1.5 Application dans le noyau : `advance_time_soa`

La version SoA introduit un noyau `advance_time_soa` (analogue à `advance_time` dans `src/ant_simu.cpp`) :

1. boucle sur $i \in [0, N)$: avance de la fourmi i en manipulant `x[i]`, `y[i]`, `state[i]`, `seed[i]` ;
2. évaporation globale des phéromones (`do_evaporation`) ;
3. validation/swap de la carte de phéromones (`update`).

La logique par fourmi reste volontairement identique à la version AoS (mêmes choix de voisin, mêmes branches, même condition de mouvement `consumed_time < 1`) afin de comparer les performances en isolant l’effet du layout mémoire.

3.1.6 Découplage de `SDL_Point` hors hot loop et adaptation du rendu

Le type `SDL_Point` est pratique pour le rendu mais il couple le noyau de calcul à SDL. Dans la stratégie SoA, le noyau manipule des scalaires `int32_t` ; la conversion vers un point SDL (si nécessaire) est repoussée au moment du rendu. Dans le même esprit, une surcharge `mark_pheronome(x,y)` côté `pheronome` permet de marquer les phéromones sans construire un `SDL_Point` à chaque pas.

Enfin, le `Renderer` est adapté pour accepter soit une population AoS (`std::vector<ant>`), soit une population SoA (`AntsSoA`). Le rendu itère alors :

- en AoS : sur les objets et `ant.get_position()` ;
- en SoA : sur l’indice i et les tableaux `x[i]/y[i]`.

3.1.7 Validation et protocole de comparaison

La validation repose sur la compilation en mode *release* puis l’exécution du mode `-benchmark` en forçant successivement `-layout aos` puis `-layout soa`. Le format des métriques `METRIC ...` est conservé afin de réutiliser le pipeline de consolidation existant (`scripts/measure_temps.sh`). La comparaison se focalise sur $K1$ (et donc $K0$), puisque la vectorisation vise prioritairement le noyau d’avance des fourmis.

3.2 Résultat du Profilage

Les résultats présentés ci-dessous proviennent de la campagne de mesure :

`results/test_vectorisation_layout_local2/soa/latest/summary.md`. La configuration utilisée est la suivante : layout `soa`, 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 6 – Temps one-shot d’initialisation (layout SoA)

Partie	ms moyen	écart-type
P0	8.189029	0.695765
P1	1.151506	0.574463
P2	0.123899	0.055120

TABLE 7 – Décomposition des coûts (profilage, layout SoA)

Partie	ms/iter moyen	écart-type	% kernel
K1	7.691928	0.284915	94.2430
K2	3.269661	0.117760	40.0605
K3	2.769702	0.146911	33.9349
K4	0.458642	0.029528	5.6194
K5	0.009918	0.001233	0.1215
R0	2.248921	0.137798	n/a
E0	0.005612	0.000447	n/a

TABLE 8 – Indicateurs globaux par répétition (layout SoA)

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	10.065456	7.899807	2.160426	632926.880286
2	10.743768	8.415819	2.321891	594119.265468
3	10.312283	8.119618	2.187337	615792.511039
4	9.861015	7.762009	2.093800	644163.165399
5	11.099146	8.611749	2.481150	580602.124544
Moyenne	10.416334	8.161800	2.248921	613520.789347
Écart-type	0.450725	0.315351	0.137798	23582.917192

Fraction parallélisable candidate (layout SoA) :

$$F = (K1 + K4)/K0 = 0.997787$$

TABLE 9 – Prédiction théorique selon Amdahl (layout SoA)

Threads	S(p)
2	1.997252
4	3.983556
8	7.923679
16	15.676438

3.3 Analyse des Résultats

Afin d’évaluer l’impact de la vectorisation SoA de manière contrôlée, nous comparons deux campagnes de mesure exécutées sur la même machine, avec les mêmes paramètres et le même script de profilage :

— AoS (baseline) : `results/test_vectorisation_layout_local2/aos/latest/summary.md` ;

— SoA (vectorisé) : `results/test_vectorisation_layout_local2/soa/latest/summary.md`.

La variation relative est définie par :

$$\Delta_{\%}(X) = 100 \times \frac{X_{\text{SoA}} - X_{\text{AoS}}}{X_{\text{AoS}}}.$$

TABLE 10 – Comparaison AoS vs SoA (moyennes sur 5 répétitions)

Métrique	AoS	SoA	Δ	$\Delta_{\%}$
T_{kernel} (ms/iter)	7.686815	8.161800	+0.474985	+6.179
$K1$ (ms/iter)	7.266526	7.691928	+0.425402	+5.854
$R0$ (ms/iter)	2.109269	2.248921	+0.139652	+6.620
$T_{\text{iter,total}}$ (ms/iter)	9.801146	10.416334	+0.615188	+6.277
fourmis/s	654695.265978	613520.789347	-41174.476631	-6.289

1. **Sur cette campagne locale, SoA est plus lent que AoS.** Les temps noyau T_{kernel} ($\approx K0$) passent de 7.686815 ± 0.631129 (AoS) à 8.161800 ± 0.315351 ms/iter (SoA), soit $\Delta_{\%} = +6.179\%$. De même, $K1$ passe de 7.266526 ± 0.571768 à 7.691928 ± 0.284915 ms/iter (+5.854%). Le changement de layout seul ne garantit donc pas un gain systématique.
2. **Hotspots stables : $K1$ reste dominant.** Dans les deux layouts, $K1$ reste proche de 94% du noyau (AoS : 94.5323%, SoA : 94.2430%). La structure de coût reste donc globalement la même, et la vectorisation de données n'a pas déplacé le goulot principal.
3. **Le coût hors noyau augmente aussi en SoA.** Le rendu $R0$ augmente de 2.109269 à 2.248921 ms/iter (+6.620%), ce qui contribue à la hausse du temps total ($9.801146 \rightarrow 10.416334$ ms/iter, +6.277%).
4. **Débit en baisse cohérente avec $K0$.** Le débit passe de 654695.265978 (AoS) à 613520.789347 fourmis/s (SoA), soit -6.289% , cohérent avec l'augmentation de T_{kernel} .
5. **Interprétation.** Cette campagne montre surtout que SoA est un *pré-requis d'architecture* (données contiguës, backend plus simple à paralléliser) plutôt qu'une garantie d'accélération immédiate en exécution séquentielle. Le gain de performance apparaît davantage dans la phase OpenMP où ce layout est exploité avec parallélisme.

4 Parallélisation en mémoire partagée (OpenMP)

4.1 Stratégie retenue

4.1.1 Objectif

Après la vectorisation (layout SoA), l'objectif du point OpenMP est de réduire le temps noyau $K0$ en parallélisant les boucles les plus coûteuses, tout en conservant une sortie benchmark reproductible (METRIC ...) pour l'analyse de speedup.

4.1.2 Choix d'architecture

L'implémentation OpenMP est portée par le backend `omp+soa` (`src/sim/omp_soa_backend.cpp`) et sélectionnée via :

- `-exec omp`
- `-layout soa`

4.1.3 Boucles parallélisées

1. **Évaporation des phéromones ($K4$).** `do_evaporation()` (`src/pheronome.hpp`) utilise `#pragma omp parallel for collapse(2) schedule(static)` sur le balayage de grille. Les cellules sont indépendantes pendant cette phase.
2. **Avance des fourmis ($K1$).** Dans `OmpSoaBackend::step`, la boucle sur les indices de fourmis est parallélisée avec `#pragma omp for schedule(static)`. Pendant la région parallèle, chaque thread :
 - met à jour ses fourmis locales (`x[i]`, `y[i]`, `state[i]`, `seed[i]`),
 - enregistre les cellules touchées (`touched_v1/touched_v2`),
 - accumule son `food_delta` local.

Aucune écriture globale directe dans la carte de phéromones n'est faite dans la boucle parallèle.

4.1.4 Correctness et absence de data races

Les points sensibles en mémoire partagée sont :

- la carte de phéromones (écritures concurrentes potentielles),
- le compteur global `food_quantity` (incréments concurrents).

Le schéma retenu évite ces conflits :

- structures locales par thread (`touched_v1/touched_v2`, `food_delta`),
- fusion après la région parallèle,
- replay séquentiel des marquages.

4.1.5 Réduction sparse et robustesse d'ordre

Après la région parallèle :

1. concaténation des vecteurs `touched` de tous les threads,
2. tri + `unique` pour ne garder qu'une fois chaque cellule,
3. application séquentielle des marques sur la carte globale.

Ce schéma reste cohérent avec un marquage idempotent (politique de fusion par max), robuste à l'ordre de traitement des fournis.

4.1.6 Profilage OpenMP-safe

Le profilage combine deux niveaux :

- **wall-time** aux frontières des kernels ($K1$, $K4$, $K5$),
- **work-time** pour $K2/K3$ (somme du travail interne des threads).

En OpenMP, $K2/K3$ sont accumulés localement par thread puis fusionnés après la région parallèle (pas d'atomiques dans la boucle chaude). Ainsi :

- $K1$ est un temps mur (comparaison directe de performance),
- $K2/K3$ décrivent la quantité de travail, mais ne doivent pas être lus comme temps mur.

4.2 Protocole expérimental

Campagnes utilisées :

- OpenMP SoA : `results/test_openmp_report_local/omp/soa/latest`
- Référence serial SoA (1 thread) : `results/test_openmp_report_local/serial/soa/latest`

Configuration commune :

- 1200 itérations, 200 warm-up, donc 1000 itérations mesurées,
- mode `-benchmark -no-render`,
- 7 répétitions par configuration,
- machine : AMD Ryzen 7 5800H (8 coeurs physiques / 16 threads logiques), WSL2,
- sweep $\{1, 2, 4, 8\}$ aligné sur les coeurs physiques.

Commande type :

```
./scripts/measure_openmp.sh --folder test_openmp_report_local --layout soa --exec omp \
  --threads 1,2,4,8 --iterations 1200 --warmup 200 --reps 7
```

4.3 Résultats

TABLE 11 – OpenMP SoA : temps noyau et speedup selon le nombre de threads

Threads	K0 moyen (ms/iter)	écart-type	Speedup $S(p)$	Efficacité $E(p) = S/p$
1	8.684056	0.264434	1.000000	1.0000
2	5.539441	0.275826	1.567677	0.7838
4	4.612187	0.373999	1.882850	0.4707
8	3.564663	0.806521	2.436151	0.3045

4.3.1 Calcul explicite du speedup mesuré

Le speedup expérimental est défini par :

$$S(p) = \frac{T(1)}{T(p)}$$

où $T(p)$ est le temps moyen de $K0$ avec p threads. Dans cette section, la référence $T(1)$ est **le backend omp+soa à 1 thread** (pas **serial+soa**), afin de mesurer strictement l'effet du threading à algorithme constant.

$$T(1) = 8.684056, \quad T(2) = 5.539441, \quad T(4) = 4.612187, \quad T(8) = 3.564663$$

$$S(1) = \frac{8.684056}{8.684056} = 1.000000$$

$$S(2) = \frac{8.684056}{5.539441} = 1.567677321 \approx 1.567677$$

$$S(4) = \frac{8.684056}{4.612187} = 1.882850308 \approx 1.882850$$

$$S(8) = \frac{8.684056}{3.564663} = 2.436151334 \approx 2.436151$$

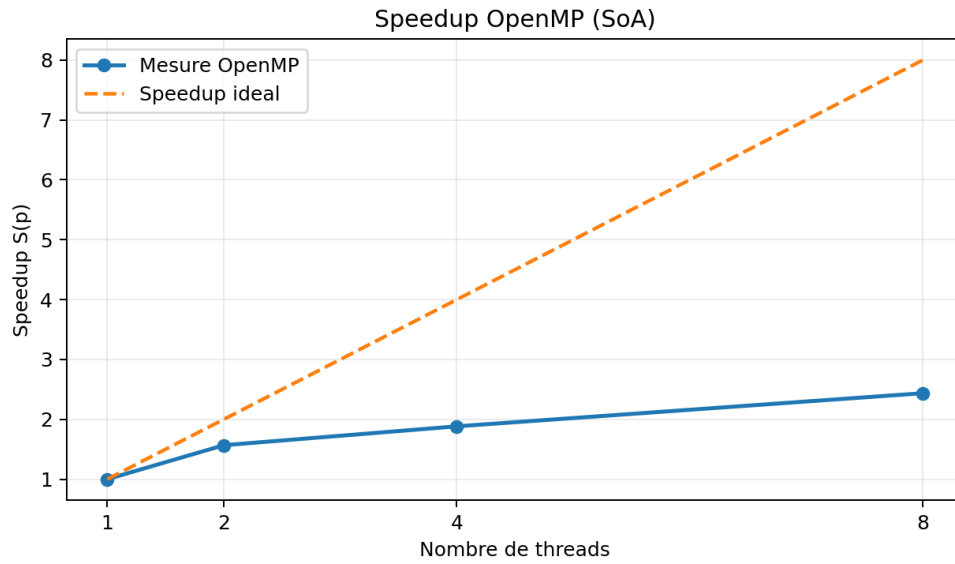


FIGURE 1 – Speedup mesuré OpenMP (SoA) vs speedup idéal linéaire

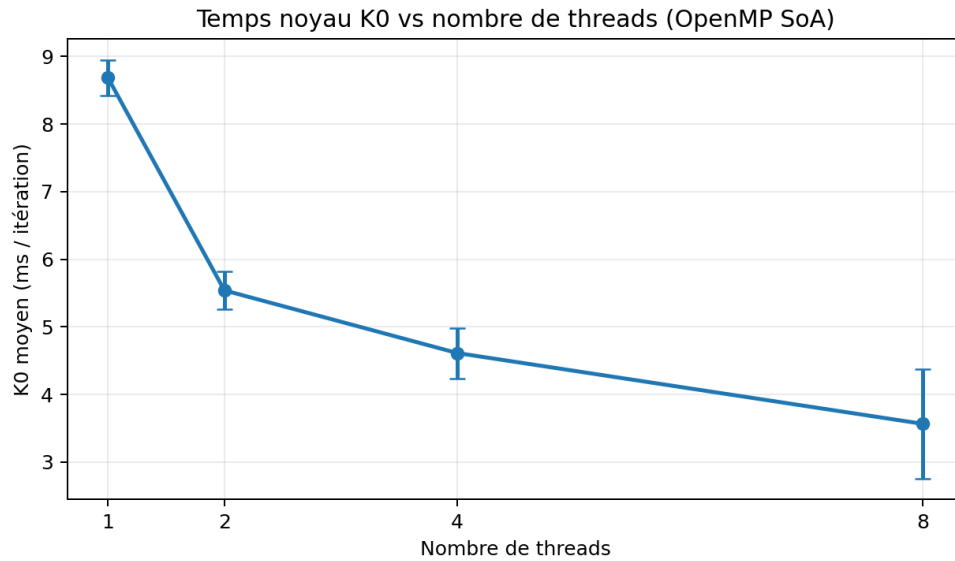
FIGURE 2 – Temps noyau $K0$ en fonction du nombre de threads (barres d'erreur : écart-type)

TABLE 12 – Décomposition du noyau OpenMP SoA (wall-time)

Threads	K1 (ms/iter)	K4 (ms/iter)	K5 (ms/iter)
1	8.014997	0.649239	0.015685
2	5.023505	0.400604	0.011876
4	4.126514	0.366018	0.014365
8	3.219344	0.317306	0.014992

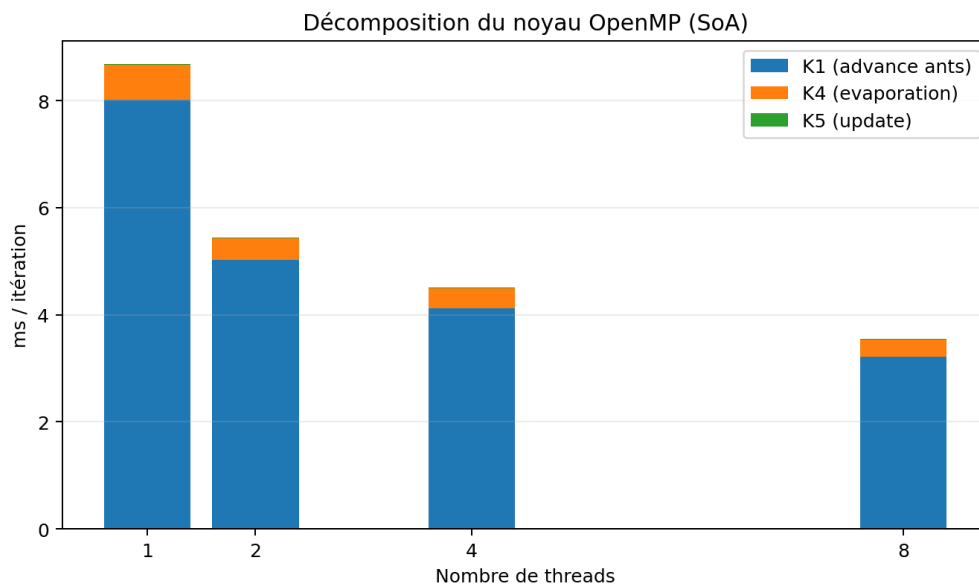
FIGURE 3 – Décomposition $K1/K4/K5$ du noyau OpenMP SoA

TABLE 13 – Work-time interne OpenMP SoA ($K2$, $K3$)

Threads	K2 work-time (ms/iter)	K3 work-time (ms/iter)
1	3.640941	0.532093
2	3.275849	0.587566
4	3.503250	0.777031
8	3.399851	0.815067

4.3.2 Vérifications empiriques de correctitude

En complément de l’argument “absence de races”, on vérifie deux invariants simples :

- `food_quantity` monotone non décroissante (`food_monotonic_ok=1`),
- stabilité d’indicateurs globaux de phéromones entre 1 et 8 threads (même seed).

TABLE 14 – Checks de correctitude OpenMP (5000 itérations, warmup 500, seed 2026)

Métrique	OMP 1 thread	OMP 8 threads
<code>food_monotonic_ok</code>	1	1
<code>food_quantity_final</code>	1556	1556
<code>phen_v1_sum</code>	2989.08	2989.08
<code>phen_v2_sum</code>	16321.2	16321.2
<code>phen_v1_max</code>	1.0	1.0
<code>phen_v2_max</code>	1.0	1.0

4.4 Analyse et discussion des résultats

Le speedup croît bien avec le nombre de threads, mais reste loin de la droite idéale :

$$S(8) = 2.4362 \quad (\text{vs idéal } 8.0)$$

- **Lecture Amdahl.** Avec $S(8) = 2.436151$, on obtient :

$$f \approx \frac{8/S(8) - 1}{7} \approx 0.33, \quad S_{\max} \approx \frac{1}{f} \approx 3.06.$$

Le plafonnement observé autour de 2.4 est cohérent avec cette borne.

- **K4 (évaporation), comportement memory-bound.**

$$K4 : 0.649239 \rightarrow 0.317306 \text{ ms/iter } (1 \rightarrow 8) \approx -51.1\%.$$

Le gain est net mais non linéaire, typique d’une limite de bande passante mémoire.

- **K1 (avance des fourmis), gain significatif mais borné.**

$$K1 : 8.014997 \rightarrow 3.219344 \text{ ms/iter } (1 \rightarrow 8) \approx -59.8\%.$$

La phase de fusion sparse (`concat`, `sort+unique`, `replay`) reste partiellement séquentielle et freine la scalabilité.

- **Interprétation de $K2/K3$.** Les valeurs $K2/K3$ sont des work-times : elles décrivent le volume de travail interne et l’évolution de la granularité, mais ne s’additionnent pas directement en temps mur.

- **Comparaison serial+soa vs omp+soa à 1 thread.** La référence serial locale vaut $K0 = 7.924494$ ms/iter, contre 8.684056 ms/iter pour **omp+soa** à 1 thread (+9.6%). Cet écart vient du coût structurel propre au backend OMP (buffers touched, merge, dedup, replay), indépendamment du speedup de threading.

Bilan.

- accélération mesurable et reproductible ($S(8) \approx 2.44$),
- $K4$ limité principalement par la mémoire,
- $K1$ limité par la fusion sparse post-parallèle.

5 Comparaison des campagnes (ancienne machine vs campagne locale)

5.1 Objectif

Les sections précédentes ont été mises à jour avec des mesures locales (cette machine). Cette section compare les indicateurs clés avec les valeurs historiques présentes dans la version précédente du rapport (campagnes d’une autre machine et d’un état de code antérieur), afin d’expliquer pourquoi certaines performances augmentent ou diminuent.

5.2 Comparaison synthétique

TABLE 15 – Écarts entre ancienne campagne et campagne locale

Indicateur	Ancienne campagne	Campagne locale	Écart
Baseline $K0$ (ms/iter)	3.979834	8.041472	+102.1%
Vectorisation $\Delta_{\%}(K0)$ SoA vs AoS	+0.034%	+6.179%	+6.145 points
OpenMP $S(8)$	1.697653	2.436151	+43.5%
OpenMP $K0@1$ (ms/iter)	2.555276	8.684056	+239.8%

5.3 Interprétation des écarts

1. **Différence de machine et d’environnement d’exécution.** Les performances absolues dépendent fortement du CPU, de la mémoire, du niveau de charge de la machine et de l’environnement (ici WSL2). Il est donc normal que les temps absolus changent sensiblement entre deux machines.
2. **Évolution du code entre campagnes.** Cette version n’est pas strictement identique à l’ancienne :
 - le backend OpenMP et le backend serial SoA partagent maintenant un coeur d’avance commun ;
 - $K2/K3$ sont activés en permanence comme *work-time*, ce qui ajoute de l’instrumentation interne et augmente les temps absolus ;
 - le backend OMP conserve un coût structurel à 1 thread (buffers touched, merge, dedup, replay).

Ces changements améliorent la comparabilité méthodologique, mais ils modifient les valeurs absolues.

3. **Différence de protocole selon les sections.** Baseline/Vectorisation sont mesurés avec rendu actif (`measure_temps.sh`), alors que la section OpenMP est mesurée en `-no-render` (`measure_openmp.sh`). Les chiffres absolus entre sections ne sont donc pas directement comparables.

5.4 Ce qui reste stable malgré les écarts

- Le hotspot principal reste $K1$ (avance des fournis).
- La vectorisation SoA seule ne garantit pas un speedup en mode séquentiel.
- OpenMP apporte un speedup net avec le nombre de threads ($S(8) > S(4) > S(2) > S(1)$).